# Solderbench: An open-source synthetic benchmark for PCB solder-joint defect inspection algorithm development

Qinmin Wang[a,1], Zixin Wang[b,2], Hongyuan Wang[a,3], Zongzhong Wang[c,*]

[a]*School of Computer Science and Artificial Intelligence, Guangdong University of Science and Technology, Dongguan, China*
[b]*School of Electronic and Communication Engineering, Tianjin Normal University, Tianjin, China*
[c]*School of Electromechanical Engineering, Guangdong University of Technology, Guangzhou, China*

**Abstract**

Automated optical inspection (AOI) of solder joints is a critical quality-control step in printed circuit board (PCB) assembly, and three-dimensional (3D) point-cloud inspection is increasingly adopted because solder quality is essentially a volumetric property. However, publicly available 3D solder-joint datasets are essentially nonexistent: industrial scan data are proprietary, so new inspection algorithms are difficult to compare fairly. We present Solderbench, an open-source, pure-Python synthetic benchmark for developing and evaluating 3D solder-joint defect inspection algorithms. Solderbench provides (i) a parametric point-cloud generator that simulates five joint classes (good, insufficient, excessive, bridging, and collapse) with more than fifteen controllable parameters and seed-level, point-wise reproducibility; (ii) a seven-dimensional geometric feature extractor; (iii) six baseline classifiers, including a re-implementation of the industrial volume-threshold rule S3DA; and (iv) automated comparison and ablation runners that export CSV, LaTeX, and figure artifacts. On a 1250-sample benchmark (seed 42), a random forest reaches 0.9440 accuracy and 0.9439 macro-F1, and feature ablation identifies height variance as the most informative descriptor. Synthetic results provide a controlled, reproducible proxy for algorithm development, but should ultimately be validated on real production scans.

*Keywords:* solder joint inspection; automated optical inspection; 3D point cloud; synthetic benchmark; defect classification; reproducible evaluation; PCB; synthetic dataset

## 1. Required metadata

## 2. Motivation and significance

SMT dominates modern printed circuit board (PCB) assembly, and solder-joint quality largely determines product reliability. AOI is the standard quality-control step on SMT lines [1, 2, 3]; classical 2D AOI and deep-learning 2D detectors [4] on public datasets such as DeepPCB [5] are mature, yet inspection is increasingly moving to 3D because solder quality is fundamentally volumetric [6, 7]. General 3D deep learning (PointNet/PointNet++ [8, 9], Open3D [10]) has advanced, but 3D solder-joint research remains hampered by the absence of public data: existing public PCB datasets are almost entirely 2D [5], and the few 3D studies rely on private, non-downloadable in-house point clouds [6, 7], so algorithms cannot be compared fairly or reproduced. Solderbench closes this gap synthetically, simulating solder-joint point clouds from a parametric geometric model whose parameters (nominal joint size, noise, defect strength, bridging distance, collapse depth) are physically interpretable and user-controllable; synthetic generation yields costless labelled data of arbitrary size, deliberate control of class difficulty and sensor noise, and seed-level reproducibility.

*Corresponding author at School of Electromechanical Engineering, Guangdong University of Technology, Guangzhou, China. E-mail: wangzz@gdut.edu.cn
[1]E-mail: wangqinmin@gdust.edu.cn
[2]E-mail: 396514839@qq.com
[3]E-mail: wanghongyuan@gdust.edu.cn

Table 1: Code metadata (mandatory).

| Nr. | Code metadata description | Value |
|---|---|---|
| C1 | Current code version | v0.1.0 |
| C2 | Permanent link to code/repository used for this code version | https://github.com/OWNER/solderbench |
| C3 | Permanent link to reproducible capsule | 10.5281/zenodo.XXXXXXX |
| C4 | Legal code license | MIT License |
| C5 | Code versioning system used | git |
| C6 | Software code languages, tools, and services used | Python ($\geq$ 3.8) |
| C7 | Compilation requirements, operating environments and dependencies | numpy, scipy, pandas, scikit-learn, matplotlib; OS-independent (no compilation, no GPU required) |
| C8 | Link to developer documentation/manual | https://github.com/OWNER/solderbench#readme |
| C9 | Support email for questions | wangzz@gdut.edu.cn |

## 3. Software description

### *3.1. Software architecture*

Solderbench is a pure-Python package (about 1,000 source lines) with five feed-forward modules behind one public API: `generator.py` (parametric simulator), `features.py` (seven descriptors), `baselines.py` (six classifiers), `evaluate.py` (`run_comparison`/`run_ablation` runners exporting CSV, booktabs LaTeX, and figures), and `data.py` (`load_dataset()`, the real-camera integration point). Stages exchange NumPy arrays / pandas DataFrames, so any stage is replaceable; synthetic clouds swap for segmented real scans at `load_dataset()` without downstream changes. The package ships a 15-test suite (shape/balance, same-seed reproducibility, feature validity, a Welch $t$-test that insufficient-solder volumes are significantly smaller, and fit/predict smoke tests), a `pyproject.toml`, and a `codemeta.json`; dependencies are only numpy, scipy, pandas, scikit-learn, and matplotlib [14, 15, 16, 17].

### *3.2. Software functionalities*

**Generation.** SolderJointGenerator simulates five SMT defect classes [6, 7]: good, insufficient (shrunk ellipsoid, 0.55–0.85 of nominal), excessive (inflated ellipsoid, mean inflation 1.3, plus a bump), bridging (small centre-to-centre distance plus a strip), and collapse (random top dent). Over fifteen parameters control $n_{\text{points}}$, noise, and the shrink/inflation/bridging/collapse ranges; class ranges deliberately overlap so no classifier is perfect, and difficulty is tuned by widening or narrowing them.

**Features.** extract_features returns seven descriptors (Table 2): `vol` (voxel volume), `max_h`, `h_var`, `area`, `vol_area_ratio`, `convex_vol_ratio` (irregularity), and `min_dist` (bridging exposure). **Baselines and**

Table 2: The seven geometric descriptors returned by `extract_features` and their physical meaning.

| Feature | Definition | Physical meaning |
|---|---|---|
| `vol` | voxel count $\times$ (voxel size)$^3$ | total solder volume |
| `max_h` | max $z$ over points | joint height |
| `h_var` | variance of point heights | height-distribution spread (dent / bump) |
| `area` | convex-hull surface area | joint surface extent |
| `vol_area_ratio` | vol / area | compactness |
| `convex_vol_ratio` | vol / ConvexHull.volume | shape irregularity (voids / dents) |
| `min_dist` | min distance to neighbouring joint | bridging exposure |

**runners.** get_baselines returns S3DA – a re-implementation of the industrial volume-threshold rule [6], threshold fixed in source and tunable – plus logistic regression, an RBF-SVM [12], kNN, gradient boosting, and a random forest [11]; all are scikit-learn estimators [13] with fixed random states. run_comparison(X, y) evaluates all six under a stratified 7:3 split with 5-fold CV, returning accuracy, macro precision/recall/F1, and CV accuracy; run_ablation(X, y) does leave-one-feature-out and single-feature analysis anchored on the random forest (Table 3, B6) to isolate descriptor contribution. Both export CSV and LaTeX, run_comparison also renders the best model's confusion matrix, and `python examples/run_benchmark.py` reproduces the full benchmark.

**Validity guarantees.** Every cloud is resampled to exactly $n_{\text{points}}$ points, preventing sampling-density leakage so volumetric features reflect shape rather than density. All randomness derives from one independent `RandomState` created at construction (not NumPy's global seed), so a given seed yields point-wise identical clouds, features, and tables – asserted by the test suite. There is no compiled code, GPU, or network access at runtime.

## 4. Illustrative examples

### *4.1. Quickstart*

Install the package and its dependencies first with `pip install -e .` (requires Python $\geq$ 3.8); the API below corresponds to version v0.1.0 and may change in later releases. The following end-to-end example generates a dataset, extracts features, and runs both experiments in eight lines:

```
from solderbench import (SolderJointGenerator, extract_features,
get_baselines, run_comparison, run_ablation)
# generate 1250 joints (5 classes x 250) at seed 42
gen = SolderJointGenerator(n_points=1500, noise=0.02, seed=42)
clouds, labels = gen.generate(n_per_class=250)   # 5 classes x 250 joints
X, y = gen.to_features(clouds, labels)           # 7-feature matrix + labels
results = run_comparison(X, y)   # 6 methods x Acc/P/R/F1 + 5-fold CV
print(results[["no", "name", "Accuracy", "F1"]])
abl = run_ablation(X, y)         # leave-one-out + single-feature dF1
print(abl.head(8))
```

### *4.2. Benchmark comparison*

`examples/run_benchmark.py` produces the benchmark snapshot reported in Table 3: 1250 samples (5 classes $\times$ 250 joints, 1500 points per cloud, seed 42, stratified 7:3 split; 5-fold cross-validation on the training set). Three observations follow. First, the industrial rule S3DA (B1) is a strong baseline at 0.8960 accuracy, confirming

Table 3: Baseline comparison on the Solderbench benchmark (seed = 42, 1250 samples). Best results in bold.

| No. | Method | Accuracy | Precision | Recall | macro-F1 | CV Acc. (5-fold) |
|---|---|---|---|---|---|---|
| B1 | S3DA (volume threshold) | 0.8960 | 0.8990 | 0.8960 | 0.8964 | $0.9234 \pm 0.0138$ |
| B2 | Logistic Regression | 0.9387 | 0.9416 | 0.9387 | 0.9386 | $0.9657 \pm 0.0198$ |
| B3 | SVM (RBF) | 0.9387 | 0.9390 | 0.9387 | 0.9385 | $0.9771 \pm 0.0177$ |
| B4 | KNN | 0.8933 | 0.8944 | 0.8933 | 0.8921 | $0.9234 \pm 0.0168$ |
| B5 | GradientBoosting | 0.9413 | 0.9417 | 0.9413 | 0.9412 | $0.9577 \pm 0.0138$ |
| **B6** | **Random Forest** | **0.9440** | **0.9452** | **0.9440** | **0.9439** | $\mathbf{0.9680 \pm 0.0160}$ |

that solder volume alone carries most of the class signal – exactly the physics the generator encodes. Second, all learning-based methods improve on the rule, with the random forest best on the held-out test set (accuracy 0.9440, macro-F1 0.9439). Third, test-set and cross-validation rankings do not fully coincide: the RBF SVM has the highest CV accuracy (0.9771) yet only 0.9387 on the held-out test set, a textbook signature of mild overfitting to the training folds. This is exactly why Solderbench reports both protocols and why users must trust the independent test set rather than cross-validation alone. Figure 1 shows the confusion matrix of the random forest on the held-out test set (375 samples, 21 misclassified). Bridging is perfectly separated (75/75), because the `min_dist` feature exposes the abnormally small inter-joint distance by construction. All 21 errors stay within the {good, insufficient, collapse} triad: collapse loses 4 samples to good and 6 to insufficient, and gains 3 good samples misclassified as collapse; good

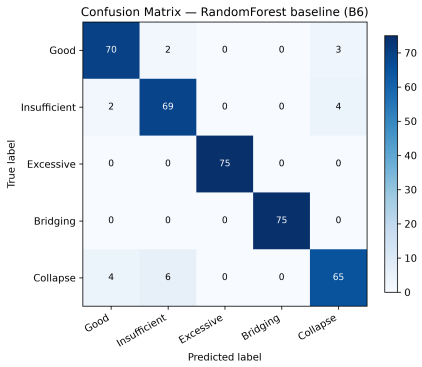



Figure 1: Confusion matrix for the Random Forest (B6) on the held-out test set (375 samples, seed = 42). Bridging is perfectly classified via the `min_dist` feature; the main confusion occurs between collapse and good/insufficient classes.

and insufficient exchange 2 samples each way, and insufficient loses a further 4 samples to collapse. The overall error therefore remains between collapse and the good/insufficient classes, because a shallow dent alters the height distribution only subtly – precisely why `h_var` – the most informative descriptor in the ablation (Table 4) – is needed to separate it; this mirrors the hard case reported for real 3D inspection systems [6, 7].

### *4.3. Feature ablation*

Table 4 reports the leave-one-feature-out (LOO) ablation for the random forest (full-model macro-F1 = 0.9439). The ablation has a clean physical reading. Height variance (`h_var`) is the single most valuable descriptor because

Table 4: Leave-one-feature-out ablation of the RandomForest model (macro-F1; full model = 0.9439).

| **Removed feature** | **dF1 vs. full model** | **Interpretation** |
|---|---|---|
| h_var | -0.0459 | Most informative: collapse dents and excessive bumps reshape the height distribution |
| vol_area_ratio | -0.0105 | Compactness cue separating volumetric defects from nominal joints |
| min_dist | -0.0054 | Small overall cost because bridging is nearly separable already |
| area | -0.0052 | Correlated with vol; adds modest independent signal |
| convex_vol_ratio | -0.0025 | Irregularity cue; partially redundant with h_var |
| vol | +0.0026 | Slightly redundant once ratios are present |
| max_h | 0.0000 | Fully redundant given h_var and vol |

the two hardest defects – collapse (a dent) and excessive solder (a bump) – are primarily height-distribution phenomena, whereas volume-only cues saturate. Single-feature runs corroborate this: the strongest single feature is `convex_vol_ratio` (macro-F1 $=$ 0.6412), which captures shape irregularity common to several defect types, while the weakest is `min_dist` (macro-F1 $=$ 0.3616), which is highly informative for bridging but carries no signal for the other four classes. The near-zero or positive deltas for `vol` and `max_h` indicate information redundancy rather than irrelevance: both are strongly correlated with the ratio features. Volume alone is in fact a strong single cue – it underpins the industrial S3DA rule (B1), which reaches 0.8960 accuracy using volume only – and its near-zero delta here reflects redundancy with the ratio features, not uselessness. Removing `vol` slightly increases macro-F1 within small numerical fluctuations; this is purely a statistical artefact given that volume information is already encoded within ratio-based descriptors.

Table 5: Selected representative single-feature macro-F1 scores (RandomForest, seed = 42). Full results are programmatically exported by run_ablation.

| Single feature | macro-F1 |
|---|---|
| `convex_vol_ratio` | 0.6412 |
| `min_dist` | 0.3616 |

## 5. Impact

Solderbench contributes to three communities. For inspection-algorithm research, it is the first public, reproducible 3D benchmark for solder-joint defect classification, replacing per-paper private datasets with a common task, protocol, and versioned reference numbers; new methods are ranked against six baselines with one command. For teaching, it is a self-contained geometric-ML case study that fits a short lab session and runs on any laptop, with each design decision doubling as a benchmark-methodology lesson. For reproducibility, seed-level determinism, a one-command script, machine-readable exports, a `codemeta.json` descriptor, and an MIT license make the software trivially citable, archivable, and reusable. The path to production data is short: a user with a real 3D AOI camera replaces the body of `load_dataset` – or passes their own X, y – with code that reads scans, segments joints, and calls `extract_features`; all downstream experiments run unchanged. Beyond the classical-ML workflow, the generator's raw point clouds can be passed directly to external deep models (e.g., PointNet [8, 9]); Solderbench supports both scenarios.

## 6. Limitations

We state the limitations of Solderbench honestly. The generator is a geometric model, not a physics simulator: it does not model wetting dynamics, surface tension, oxidation, or realistic sensor artifacts (occlusion, multi-path reflections, structured-light failure), and its noise is a simple isotropic perturbation; class distributions are plausible and overlapping, not fitted to production data. Three scope limits remain. First, the five classes are idealized single-fault geometries – compound defects (e.g., insufficient and collapsed together) are unsupported, although they occur on real lines. Second, the clouds model only the individual joint, omitting substrate, pads, and pins, so board-level context is out of scope. Third, resampling every cloud to a fixed $n_{\text{points}}$ simplifies real sensors, which return a variable count, and may alter the point-density distribution. **Consequently, Solderbench is suitable for algorithm prototyping, fair cross-method comparison, and descriptor-level analysis, but its metric values must not be read as predictions of real industrial AOI performance.** Synthetic validation cannot substitute for validation on real scans; the benchmark guarantees only that comparisons made on it are fair, controlled, and exactly reproducible – the property the field currently lacks.

**Data availability.** The Solderbench source code and all reproduction scripts are released under the MIT license and archived on Zenodo (doi:`10.5281/zenodo.XXXXXXX`); the permanent GitHub repository is `https://github.com/OWNER/solderbench`. The benchmark dataset is not stored as a fixed file but generated deterministically at runtime by running `python examples/run_benchmark.py`, which regenerates every table and figure in this paper.

## 7. Conclusions

We presented Solderbench, an open-source, pure-Python synthetic benchmark for 3D PCB solder-joint defect inspection with a parametric five-class generator, seed-level reproducibility, a seven-dimensional feature extractor, six baselines including the industrial S3DA rule, and automated comparison/ablation runners. On the reference benchmark (1250 samples, seed 42) a random forest reaches 0.9440 accuracy and 0.9439 macro-F1. By removing the data-access barrier that fragments 3D solder-joint inspection, Solderbench makes algorithm development, comparison, and teaching open and reproducible. Future work: (i) fitting generator parameters to measured scans; (ii) adding sensor-artifact models; (iii) compound-defect taxonomy; (iv) real-data conversion scripts; (v) deep point-cloud model interfaces (e.g. PointNet, PointNet++); and (vi) preset difficulty templates.

## 8. Acknowledgements

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors. Computations were performed with the Solderbench toolkit (MIT license).

### Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

### CRediT authorship contribution statement

**Qinmin Wang:** [TODO: CRediT roles] **Zixin Wang:** [TODO: CRediT roles] **Hongyuan Wang:** [TODO: CRediT roles] **Zongzhong Wang:** [TODO: CRediT roles – corresponding author]

### References

[1] Q. Ling, N.A.M. Isa, Printed circuit board defect detection methods based on image processing, machine learning and deep learning: A survey, IEEE Access 11 (2023) 15921-15944. https://doi.org/10.1109/ACCESS.2023.3245093

[2] Y. Zhou, M. Yuan, J. Zhang, G. Ding, S. Qin, Review of vision-based defect detection research and its perspectives for printed circuit board, Journal of Manufacturing Systems 70 (2023) 557-578. https://doi.org/10.1016/j.jmsy.2023.08.019

[3] M. Moganti, F. Ercal, Automatic PCB inspection systems, IEEE Potentials 14(3) (1995) 6-10. https://doi.org/10.1109/45.464686

[4] W. Dai, A. Mujeeb, M. Erdt, A. Sourin, Soldering defect detection in automatic optical inspection, Advanced Engineering Informatics 43 (2020) 101004. https://doi.org/10.1016/j.aei.2019.101004

[5] S. Tang, F. He, X. Huang, J. Yang, Online PCB defect detector on a new PCB defect dataset, arXiv preprint arXiv:1902.06197 (2019). https://doi.org/10.48550/arXiv.1902.06197

[6] Y. Xu, Z. Wang, J. Liang, Y. Zhang, J. Feng, W. Miao, J. Wang, G. Pu, S3DA: A 3D point cloud based PCB solder defect detection algorithm, in: S. Liu (Ed.), Software Fault Prevention, Verification, and Validation (SFPVV 2024), Lecture Notes in Computer Science, vol. 15393, Springer, 2025, pp. 115-130. https://doi.org/10.1007/978-981-96-1621-3_8

[7] Q. Hu, K. Hao, B. Wei, H. Li, An efficient solder joint defects method for 3D point clouds with double-flow region attention network, Advanced Engineering Informatics 52 (2022) 101608. https://doi.org/10.1016/j.aei.2022.101608

[8] C.R. Qi, H. Su, K. Mo, L.J. Guibas, PointNet: Deep learning on point sets for 3D classification and segmentation, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2017, pp. 652-660. https://doi.org/10.1109/CVPR.2017.16

[9] C.R. Qi, L. Yi, H. Su, L.J. Guibas, PointNet++: Deep hierarchical feature learning on point sets in a metric space, in: Advances in Neural Information Processing Systems 30 (NeurIPS), 2017, pp. 5099-5108. https://proceedings.neurips.cc/paper/2017/hash/d8bf84be3800d12f74d8b05e9b89836f-Abstract.html

[10] Q.-Y. Zhou, J. Park, V. Koltun, Open3D: A modern library for 3D data processing, arXiv preprint arXiv:1801.09847 (2018). https://arxiv.org/abs/1801.09847

[11] L. Breiman, Random forests, Machine Learning 45(1) (2001) 5-32. https://doi.org/10.1023/A:1010933404324

[12] C. Cortes, V. Vapnik, Support-vector networks, Machine Learning 20(3) (1995) 273-297. https://doi.org/10.1007/BF00994018

[13] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, E. Duchesnay, Scikit-learn: Machine learning in Python, Journal of Machine Learning Research 12 (2011) 2825-2830.

[14] C.R. Harris, K.J. Millman, S.J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N.J. Smith, et al., Array programming with NumPy, Nature 585 (2020) 357-362. https://doi.org/10.1038/s41586-020-2649-2

[15] P. Virtanen, R. Gommers, T.E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, et al., SciPy 1.0: Fundamental algorithms for scientific computing in Python, Nature Methods 17 (2020) 261-272. https://doi.org/10.1038/s41592-019-0686-2

[16] W. McKinney, Data structures for statistical computing in Python, in: Proceedings of the 9th Python in Science Conference, 2010, pp. 51-56. https://doi.org/10.25080/Majora-92bf1922-00a

[17] J.D. Hunter, Matplotlib: A 2D graphics environment, Computing in Science & Engineering 9(3) (2007) 90-95. https://doi.org/10.1109/MCSE.2007.55